

Chapter 13

클러스터 Node와 Network 보안

Kubernetes in Action

hostNamespaces · SecurityContext · PSA · NetworkPolicy

목차

13장 전체 흐름

13.1

Pod에서 호스트 Node의 Namespace 사용하기

hostNetwork · hostPort · hostPID · hostIPC

13.2

컨테이너의 보안 컨텍스트 설정

runAsUser · runAsNonRoot · capabilities · seccomp · AppArmor

13.3

Pod의 보안 관련 기능 사용 제한 (PSA)

Pod Security Admission · 3가지 프로파일 · 3가지 모드

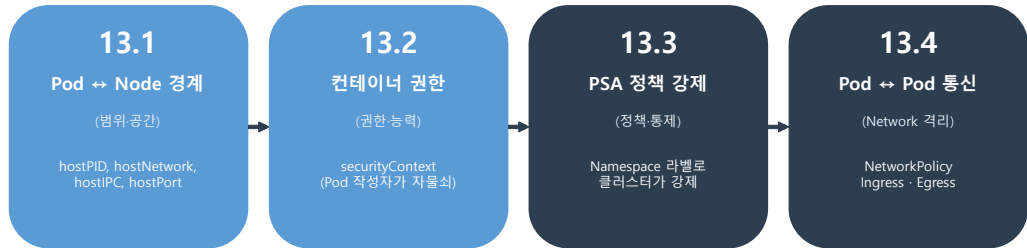
13.4

Pod Network 격리 (NetworkPolicy)

podSelector · namespaceSelector · ipBlock · Egress · ANP

13장 전체 흐름

네 절은 하나의 보안 모델을 이룬다



"Pod 하나"를 단단히 → "Pod 사이"의 통로까지 제어

13.1~13.3은 Pod 단위의 보안. 13.4에서 시선이 Pod들 사이로 확장됩니다.

Section 13.1

Pod에서 호스트 Node의 Namespace 사용하기

Pod의 독방 벽을 허물고 Node 자원에 접근하기

13.1 들어가기 전에

Pod는 원래 "독방"이다

Pod는 격리되어 있다

- 자기만의 Network 인터페이스
- 자기만의 프로세스 목록 (PID)
- 자기만의 호스트네임
- Node에서 무슨 일이 일어나는지 모름

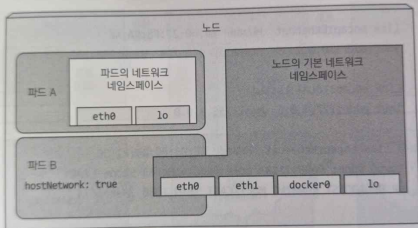
host* 스펙: 격리를 벗어나 Node에 접근

- **hostNetwork** — Node Network 사용
- **hostPID** — Node 프로세스 보기
- **hostIPC** — Node IPC 채널 사용
- **hostPort** — Node 특정 포트 점유

* Pod가 포함되었던 Node까지 접근

프로세스 식별 번호(PID, Process ID): 운영체제에서 실행 중인 각각의 프로세스를 고유하게 식별하기 위해 부여하는 식별 번호
프로세스 간 통신 (IPC, Inter-Process Communication): 운영체제에서 독립된 프로세스들이 서로 데이터를 주고받거나 자원을 공유하는 메커니즘

출처: kubernetes.io/docs/concepts/security/pod-security-standards/



▲ 그림 13.1 hostNetwork: true로 설정한 파드는 파드 네트워크 인터페이스 대신 노드 네트워크 인터페이스를 사용한다.

다음 예제는 노드의 네트워크 인터페이스를 사용하는 파드 매니페스트다. 실행해보자.

이 옵션들은 Pod 격리라는 쿠버네티스의 기본 안전장치를 일부러 깨는 행위입니다. 편리하지만, 침해당했을 때 Node 전체가 위험에 빠질 수 있는 강력한 권한이므로, 꼭 필요한 시스템 Pod에만 신중하게 사용해야 합니다

13.1.1 hostNetwork

Node의 **Network Namespace** 그대로 사용

기본 (hostNetwork: false)

```
# Pod 안에서 ip addr
eth0: 10.244.0.5
    ↑ 쿠버네티스 가상 IP
lo: 127.0.0.1
```

- 컨테이너 가상 인터페이스만 보임
- Pod 간 통신은 가상 Network 경유
- 같은 Node의 80 포트와 충돌 없음

hostNetwork: true

```
# Pod 안에서 ip addr
eth0: 192.168.1.10
    ↑ Node의 실제 NIC!
lo: 127.0.0.1
```

- Node IP로 직접 바인딩 가능
- Node의 열린 포트에 직접 접근

13.1.1 hostNetwork

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-host-network
spec:
  hostNetwork: true
  containers:
  - name: main
    image: alpine
    command: ["/bin/sleep", "999999"]
```

비유

보통은 아파트 각 세대(Pod)가 자기 현관과 자기
우편함을 갖지만,

hostNetwork는 건물 1층 로비(Node)를 통째로 내 거실로
쓰는 것과 같습니다.

실제 사용처

kube-proxy, node-exporter 등 Node Network를 직접 다뤄야 하는 시스템 Pod

13.1.2 hostPort

Node의 특정 포트 하나만 노출

```
spec:
  containers:
  - image: luksa/kubia
    name: kubia
    ports:
    - containerPort: 8080
      hostPort: 9000    # Node:9000 → Pod:8080
```

📌 핵심

- Node Network 전체가 아니라 딱 하나의 포트만 노출
- Node 9000 포트에 들어온 트래픽이 곧장 이 Pod로 전달

hostPort vs NodePort 비교

구분	hostPort	NodePort 서비스
연결 대상	Pod가 떠 있는 그 Node의 포트만	모든 Node의 같은 포트
트래픽 경로	Node → Pod (직통)	어느 Node든 → 서비스 → Pod 분산
Pod 개수	Node당 1개 (포트 중복 불가)	여러 Pod 로드밸런싱

📖 출처: kubernetes.io/docs/concepts/workloads/pods/#pod-networking

13.1.3 hostPID

Node 프로세스 목록 노출

기본 (hostPID: false)

```
# Pod 안에서 ps aux
PID  COMMAND
1    /bin/sleep 999999
↑ 자기 컨테이너 내부만
```

- 자기 컨테이너 프로세스만 보임
- Node에서 돌고 있는 다른 프로세스는 존재 자체를 모름

hostPID: true

```
# Pod 안에서 ps aux
PID  COMMAND
1    /sbin/init
2    kubelet
3    containerd
...  Node 모든 프로세스
```

- Node 모든 프로세스가 보임
- 환경변수(/proc/<pid>/environ) 접근 시도 가능
- 시스템 자원(/proc/<pid>/mem) 접근 시도 가능
(실제 접근 가능 여부는 인증/인가 확인)

13.1.3 hostIPC

컨테이너가 호스트(실제 서버) 컴퓨터의 IPC(프로세스 간 통신) 자원을 공유하도록 허용하는 설정

💡 IPC란?

프로세스끼리 데이터를 주고받는 채널.

- 기본적으로 컨테이너는 독립된 방(네임스페이스)에 격리되어 있어, 호스트나 다른 컨테이너의 메모리를 볼 수 없습니다.
- hostIPC: true를 켜면 컨테이너가 격리 벽을 깨고 나와 호스트 컴퓨터의 공유 메모리 영역에 직접 참여하게 됩니다.

```
# hostIPC: true 일 때 ipcs
----- Shared Memory -----
shmid  owner  size
0x0001  root   4096
    ↑ 노드의 다른 프로세스가 올려둔 공유 메모리
```

위험

- 목록만 보는 게 아닙니다.
- 권한이 맞으면 공유 메모리를 직접 읽거나 덮어쓸 수 있음
- DB가 공유 메모리 캐시를 쓰면 그 내용에 접근 가능

실전 예시 — kube-proxy

hostNetwork: true가 없으면 iptables 규칙이 엉뚱한 곳에 쓰인다

```
spec:
  hostNetwork: true           # Node iptables 조작 필수
  containers:
  - name: kube-proxy
    securityContext:
      privileged: true        # CAP_NET_ADMIN 등
    command:
    - /usr/local/bin/kube-proxy
```

동작 흐름

- ① kube-proxy → kube-apiserver
watch: Service-Endpoint
- ② 변경사항 수신
"ClusterIP → Pod IP"
- ③ Node iptables에 규칙 작성
← hostNetwork 필요

hostNetwork 없으면?

iptables 규칙이 Pod 내부 Namespace에만 적용 →
Service IP 라우팅이 전혀 안 됨 → 클러스터 네트워킹 전체가 멈춤

실전 예시 — node-exporter

hostPID: true가 없으면 Node 메트릭을 못 읽는다

```
spec:
  hostPID: true      # /proc 전체 접근
  hostNetwork: true  # Node NIC 통계
  containers:
  - name: node-exporter
    volumeMounts:
    - name: proc
      mountPath: /host/proc
      readOnly: true
  volumes:
  - name: proc
    hostPath:
      path: /proc
```

/proc 에서 수집

/proc/<pid>/stat
→ CPU 사용량

/proc/<pid>/status
→ 메모리 사용량

/proc/<pid>/io
→ I/O 통계

→ hostPID 없으면 자기 Pod만 보여서
Node 메트릭이 0에 가까움

💡 두 Pod 모두 DaemonSet으로 배포 — Node마다 1개씩 자동 배치

Section 13.2

컨테이너의 보안 컨텍스트 설정

각 컨테이너의 자물쇠 — Linux 실행 권한 조절

13.1과 13.2는 서로 다른 축을 조절한다

범위(어디까지 보이냐) vs 권한(무엇을 할 수 있냐)

13.1 — 범위·공간

"어디까지 볼 수 있냐"

Node와의 경계.
host* 옵션으로 Namespace 공유.

13.2 — 권한·능력

"무엇을 할 수 있냐"

컨테이너 실행 권한.
securityContext로 조절.

host* 옵션은 Node의 Network Namespace 사용했다면,
13.2에서는 접근한 Node의 Network Namespace 에서 특정 사용자로 권한(securityContext)을 설정할 수 있게됩니다.

13.1 vs 13.2 — YAML 위치

```
spec:
  hostPID: true           # pec 최상위 정의
  hostNetwork: true      # Node Namespace 공유

  containers:
    - name: app
      securityContext:    # 컨테이너별
        runAsNonRoot: true # Linux 실행 권한 조절
        capabilities:
          drop: [ALL]
```

Pod 수준

spec.securityContext
→ 모든 컨테이너의
기본값

컨테이너 수준

spec.containers[]
 .securityContext
→ 해당 컨테이너에만
Pod 수준 덮어씀

왜 사용하는 지?

13.1이 만드는 위험

hostPID: true → Node 프로세스 노출
hostNetwork: true → Node 포트·NIC 접근

+

이미지 기본값이 root

컨테이너 프로세스가
Node의 root와 동일한 권한



결과 — Node 장악

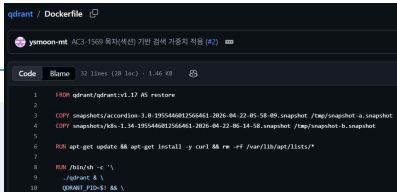
- 보이는 프로세스에 kill 시그널 전송, /proc/*/environ 읽기 (환경변수 탈취)
- 열린 포트에 바인딩, iptables 조작 (트래픽 가로채기)
- 공유 메모리 변조 (다른 프로세스의 데이터 조작)
- 결과적으로 Node 전체 장악

그 전에 왜 사용하는 지?

왜 컨테이너 이미지는 기본적으로 root로 실행되나?

1. Dockerfile USER 미지정

USER 지시문이 없으면 기본값이 root.
패키지 설치(`apt-get`)에 편하기 때문.



```
qdrant / Dockerfile
ysmoon-mit AC3-1569 목차(색션) 기반 검색 가중치 적용 (#2)
Code Blame 32 lines (28 loc) · 1.46 KB
1 FROM qdrant/qdrant:v1.17 AS restore
2
3 COPY snapshots/accordion-3.0-1955460812566461-2026-04-22-05-58-09.snapshot /tmp/snapshot-a.snapshot
4 COPY snapshots/kds-1.34-1955460812566461-2026-04-22-06-14-58.snapshot /tmp/snapshot-b.snapshot
5
6 RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
7
8 RUN /bin/sh -c '\
9   ./qdrant & \
10  QDRANT_PID=$! && \
```

2. non-root 빌드 작업 생략

useradd + chown + USER 추가 작업이 필요해서
많은 이미지가 그냥 root 그대로.

3. 1024 미만 포트 바인딩

80-443 등 well-known 포트는 root만 바인딩 가능.
웹 서버 관행이 이어진 것.

△ 핵심 — 컨테이너 안 root는 Node의 root와 같은 UID. 격리가 깨지는 순간 Node root가 되버림.

13.2.1~13.2.3 runAs* — 누구로 실행할 것인가

특정 UID·GID 강제 또는 root 차단

```
# 13.2.1 runAsUser — 특정 UID
spec:
  containers:
    - securityContext:
        runAsUser: 405

# 13.2.2 runAsGroup — 기본 GID
securityContext:
  runAsUser: 1000
  runAsGroup: 3000

# 13.2.3 runAsNonRoot — root만 아니면 됨
securityContext:
  runAsNonRoot: true
```

runAsUser

이미지가 박아둔 사용자가 아니라
내가 지정한 **UID**로 실행.

runAsGroup

기본 GID 지정. 다른 UID라도
같은 **GID**면 그룹 권한으로 접근.

runAsNonRoot

root(UID 0)로 시작하려 하면 컨테이너 자체가 거부됨.
--> **root만** 아니면 됨

13.2.4 allowPrivilegeEscalation

실행파일 내부에서 권한을 얻는 것을 거부함.

```
securityContext:  
  allowPrivilegeEscalation: false  
  
# allow(알로우): 허락  
# Privilege(프리빌리지): 특정 권한  
# Escalation(에스컬레이션): 단계적 확대
```

특정 실행파일(setuid 비트)은 파일 소유자의 권한으로 실행됨.
예: /usr/bin/passwd → root 권한 획득.

13.2.5 privileged(프리빌리지드)

Node의 모든 장치·커널 기능에 대한 전권 — 가장 위험

```
securityContext:  
  privileged: true
```

🔑 마스터키 — 모든 문을 여는 권한

```
# 일반 컨테이너 ls /dev  
null random urandom zero ...  
→ 가상 장치만
```

```
# privileged ls /dev  
sda sda1 sdb nvme0 tty ...  
→ Node 실제 디스크 전부
```

특권 모드에서 가능해지는 것

/dev/sda 직접 읽기·쓰기	→ Node 디스크 전체 접근
iptables 조작	→ Node Network 규칙 변경
커널 모듈 로드 (insmod)	→ 커널 자체 변조
mount 명령	→ Node 파일시스템 마운트

너무 권한이 많아...

13.2.6 Capabilities(케이퍼빌리티즈)

전권(privileged) 대신 필요한 능력(권한)만 골라서

Linux Capabilities — root 전권 → 약 40개의 작은 기능으로 분할

전통적으로 "일반 사용자 vs root(전권)"의 이분법이었음.
"시간 변경만 필요한 프로세스에도 root가 필요"한 문제를 해결하기 위해 도입.

Capability	의미
CAP_NET_BIND_SERVICE	1024 미만 포트 바인딩 (80, 443)
CAP_SYS_TIME	시스템 시간 변경
CAP_CHOWN	파일 소유자 변경
CAP_NET_ADMIN	Network 설정 변경 (iptables)
CAP_SYS_ADMIN	사실상 root에 준하는 광범위 권한 (위험)

13.2.6 Capabilities — 세 가지 사용 패턴

방향 1 — 제거(drop) 위주

```
capabilities:  
  drop: [SYS_ADMIN,  
         NET_ADMIN]
```

기존 root 관행 유지하면서
위험한 것만 제거

방향 2 — 부여(add) 위주

```
runAsNonRoot: true  
capabilities:  
  add: [NET_BIND_SERVICE]
```

non-root로 시작 +
꼭 필요한 것만 부여

방향 3 — drop ALL + 선별

```
runAsNonRoot: true  
capabilities:  
  drop: [ALL]  
  add: [NET_BIND_SERVICE]
```

PSA restricted 권장.
가장 안전한 패턴

13.2.7 readOnlyRootFilesystem

컨테이너 루트 파일시스템을 읽기 전용으로 잠그기

```
securityContext:
  readOnlyRootFilesystem: true

volumeMounts:
- name: tmp
  mountPath: /tmp
- name: logs
  mountPath: /var/log/app
```

왜 쓰는가

- 공격자가 악성 바이너리를 심을 수 없음 (지속성 차단)
- 컨테이너 불변성 보장
→ 감사·디버깅 신뢰성

애플리케이션이 구동되려면 로그를 쌓거나 **임시 파일(Tmp)**을 써야 하는 경우가 많습니다. `readOnlyRootFilesystem: true`를 켜면 이 작업들이 모두 실패하며 **에러가 발생합니다.**

💡 쓰기가 필요한 경로는 볼륨으로 해결

루트 파일시스템 전체는 잠그되, /tmp·로그 등 쓰기 필요 경로만 `emptyDir` 볼륨으로 열어주는 방식. 도서관 책장 잠그고 메모지만 따로 두는 것과 같음.

13.2.8 fsGroup, supplementalGroups

다른 UID 컨테이너들이 같은 볼륨 공유

```
spec:
  securityContext:
    fsGroup: 555
    supplementalGroups:
      - 666
      - 777
```

왜 필요한가

다른 UID로 실행되는 여러 컨테이너가
같은 볼륨을 공유하면 권한 충돌.
→ 공용 그룹으로 묶어 해결.

동작 — fsGroup: 555

- 볼륨 파일 소유권이 GID 555로 설정됨
- 컨테이너 A (UID 1000)가 GID 555 그룹에 추가 → 그룹 권한으로 접근
- 컨테이너 B (UID 2000)가 GID 555 그룹에 추가 → 동일하게 접근

 1.31+ — supplementalGroupsPolicy: Strict (이미지 /etc/group 무시)

13.2.9 seccompProfile

프로세스가 호출할 수 있는 syscall을 화이트리스트로 제한 (1.19 stable)

syscall이란?

프로세스가 커널에 "이것 좀 해줘"라고 요청하는 유일한 통로.
open, read, write, socket, fork, execve, kill, ptrace, mknod 등 약 300여 개.

```
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault
```

type	의미
Unconfined	제한 없음 (기본값, 비권장)
RuntimeDefault	런타임 제공 기본 프로파일 (권장)
Localhost	Node의 커스텀 JSON 프로파일

💡 RuntimeDefault — 일반 앱에 불필요한 위험한 syscall 수십 개 차단

PSA restricted는 type이 RuntimeDefault 또는 Localhost일 것을 강제.
Unconfined 상태로 restricted Namespace에 Pod를 만들면 거부됨.

13.2.10 appArmorProfile

프로그램(실행 파일) 단위로 파일·Network 접근 제한 (1.30 beta, 1.31 GA)

seccomp vs AppArmor

seccomp → "이 컨테이너는 open·read·write syscall만 쓸 수 있다" (컨테이너 전체 syscall)

AppArmor → "nginx 실행 파일은 /var/www/html만 읽을 수 있다" (실행 파일별 경로)

```
# Localhost 프로파일 예시
profile my-nginx flags=(...) {
    /var/www/html/** r,
    /var/log/nginx/** w,
    network tcp,

    deny /etc/passwd r,
    deny /proc/** r,
}
```

```
# Pod 스펙
containers:
- name: main
  image: nginx
  securityContext:
    appArmorProfile:
      type: Localhost
      localhostProfile:
        my-nginx
```

13.2 한 컨테이너의 자물쇠 모음

옵션을 익혔으니 13.1과 조합한 결과를 보자

목표	옵션	버전
누구로 실행?	runAsUser, runAsNonRoot, runAsGroup	stable
권한 크기	privileged, capabilities.add/drop	stable
파일 변조 방지	readOnlyRootFilesystem	stable
볼륨 공유 권한	fsGroup, supplementalGroups, supplementalGroupsPolicy	stable / 1.31+
시스템 콜 제한	seccompProfile	1.19 stable
프로그램 접근 제한	appArmorProfile	1.31 stable

⚠ 한계 — 양심에 달려 있다

자물쇠를 거는 건 어디까지나 Pod 작성자의 양심.
→ 13.3 PSA에서 "애초에 위험한 옵션을 못 쓰게 클러스터가 강제".

📖 출처: kubernetes.io/docs/tasks/configure-pod-container/security-context/

Section 13.3

Pod의 보안 관련 기능 사용 제한 (Pod Security Admission, PSA)

양심이 아니라 규칙으로 막기 — 클러스터가 강제

13.1 → 13.2 → 13.3 흐름

세 절은 하나의 이야기로 이어진다

13.1

Pod ↔ Node 경계

hostPID, hostNetwork, hostIPC

↓ "근데 볼 수 있어도 root면 다 할 수 있잖아"

13.2

컨테이너 권한

runAsNonRoot, capabilities, privileged...

↓ "근데 작성자가 privileged: true 써버리면 못 막잖아"

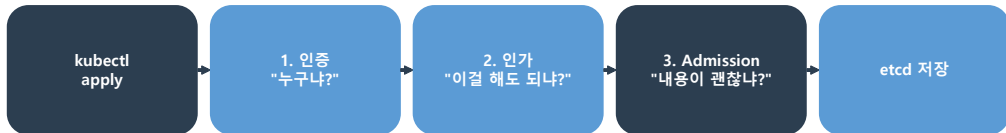
13.3

클러스터 강제

Pod Security Admission (PSA)

API 서버의 요청 처리 흐름

PSA는 Admission Control 단계에서 동작



PSA의 위치 — Admission Control 단계, API 서버에서 막음 (Node 아님)

- Validating Admission Controller로 동작
- 거부 시 etcd에 저장조차 안 됨
- 스케줄러도, Node도, kubelet도 Pod의 존재를 모름
- 즉 위반 Pod는 "클러스터에 존재하지 않는 상태"가 됨

어떤 상황에서 PSA가 필요한가

파드 내부에서 설정하는 과정에서 실수를 방지

상황	PSA가 하는 일
개발자 실수	로컬 테스트용 <code>privileged: true</code> 가 운영에 배포되려 할 때 API 서버가 즉시 거부
멀티팀 클러스터	네임스페이스별로 다른 프로파일 적용 — 팀별 보안 수준 분리
보안 컴플라이언스	PCI-DSS, CIS Benchmark 조건을 <code>restricted</code> 로 자동 충족 + 감사 증적 확보
신규 클러스터	<code>AdmissionConfiguration</code> 으로 모든 네임스페이스에 기본 프로파일 설정

13.3은 "어떤 보안 설정을 가진 Pod를 이 Namespace에 생성할 수 있는가"를 클러스터가 강제하는 장입니다.

13.3.1 PSA의 세 가지 프로파일

직접 규칙을 짤 필요 없이 이름만 고르면 됨

1. Privileged (특권)

가장 느슨한, 사실상 제한이 거의 없는 프로파일입니다. 알려진 모든 권한 상승을 허용합니다.

- 모든 것이 허용됨 — 호스트 네임스페이스 공유, 특권 컨테이너, 임의의 `capability` 추가 등 전부 가능
- 시스템 컴포넌트나 인프라 워크로드처럼 신뢰할 수 있고 높은 권한이 꼭 필요한 경우에 사용
- 예: CNI 플러그인, 스토리지 드라이버, 로그 수집기 같은 권한이 필요한 시스템 레벨 파드

라벨을 안 붙이면 기본값이 사실상 `privileged` 로 동작합니다.

13.3.1 PSA의 세 가지 프로파일

직접 규칙을 짤 필요 없이 이름만 고르면 됨

2. Baseline (기준)

알려진 권한 상승을 막는 최소한의 방어선입니다. 일반적인 애플리케이션이 큰 수정 없이 동작할 수 있는 수준이라 가장 적용하기 무난합니다.

주요 금지 항목:

- 특권 컨테이너(`privileged: true`) 금지
- 호스트 네임스페이스(`hostNetwork`, `hostPID`, `hostIPC`) 공유 금지
- `hostPath` 볼륨 금지
- 위험한 `capability` 추가 금지 (`NET_RAW` 등 기본 셋 외 추가 제한)
- HostPorts 사용 제한
- 호스트의 위험한 경로/프로세스 접근 차단

요약하면 "호스트를 건드리거나 특권을 얻는 행위"를 막는 수준입니다.

13.3.1 PSA의 세 가지 프로파일

직접 규칙을 짤 필요 없이 이름만 고르면 됨

3. Restricted (제한)

현재 권장되는 파드 강화(**hardening**) 모범 사례를 강제하는 가장 엄격한 프로파일입니다. 보안을 중시하는 운영 환경에 적합하지만, 애플리케이션이 이 요건에 맞게 작성되어 있어야 통과합니다.

Baseline의 모든 제약에 더해:

- `runAsNonRoot: true` — 루트로 실행 금지
- 권한 상승 금지 (`allowPrivilegeEscalation: false`)
- 모든 `capability`를 drop하고 (`drop: ["ALL"]`), `NET_BIND_SERVICE` 만 예외적으로 추가 허용
- `seccompProfile` 을 `RuntimeDefault` 또는 `Localhost` 로 명시적으로 설정
- 허용된 볼륨 타입만 사용 (`configMap`, `emptyDir`, `projected`, `secret`, `PVC` 등)

비교 요약

프로파일	엄격도	대표 용도
privileged	제한 없음	신뢰된 시스템/인프라 워크로드
baseline	중간	일반 애플리케이션, 무난한 기본 보안
restricted	엄격	보안 강화가 필요한 운영 워크로드

13.3.3 Namespace에 적용하기

라벨 하나가 그 Namespace 안 모든 Pod에 자동 적용

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-app
  labels:
    pod-security.kubernetes.io/enforce: restricted      # 위반 시 생성 거부
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: restricted         # 감사 로그 기록
    pod-security.kubernetes.io/audit-version: latest
    pod-security.kubernetes.io/warn: restricted          # 경고 메시지 출력
    pod-security.kubernetes.io/warn-version: latest
```

모드	동작	실전 용도
<code>enforce</code>	위반 시 파드 생성 거부	실제 차단
<code>audit</code>	위반해도 생성은 되지만 감사 로그에 기록	위반 현황 파악
<code>warn</code>	위반해도 생성은 되지만 kubectl에 경고 메시지 출력	개발자에게 즉각 알림

라벨 형식

pod-security.
kubernetes.io/
<모드>: <프로파일>

-version 라벨

latest — 클러스터
버전을 따라감

v1.31 — 특정 버전
기준에 고정
(업그레이드 시
정책 변동 방지)

⚠ 라벨을 붙인 이후 새로 생성되는 Pod에만 적용. 이미 실행 중이던 Pod는 유지.

13.3.4 점진적 도입 패턴

기존 클러스터에 `enforce: restricted`를 바로 걸면 위험

기존 클러스터에 `enforce: restricted`를 바로 걸면 기존 파드들이 한꺼번에 거부될 수 있습니다.

1단계 — warn-audit만 걸기

```
labels:  
  pod-security.kubernetes.io/warn: restricted  
  pod-security.kubernetes.io/audit: restricted
```

파드는 정상 생성되지만 위반 목록이 경고와 감사 로그에 쌓입니다.

2단계 — 위반 파드 수정

```
kubectl get events -n my-app | grep PodSecurity
```

3단계 — enforce 적용

```
labels:  
  pod-security.kubernetes.io/enforce: restricted  
  pod-security.kubernetes.io/warn: restricted  
  pod-security.kubernetes.io/audit: restricted
```

PSA는 라벨을 붙인 이후에 새로 생성되는 파드에만 적용됩니다. 이미 실행 중이던 파드는 그대로 유지됩니다.

결론 — "Pod 하나"를 보호하는 세 겹이 완성

13.1~13.3을 합치면 Pod 단위 보안 완성

절	보호 범위	통제 주체	통제 방식
13.1	Pod ↔ Node 경계	Pod 작성자	hostPID, hostNetwork 직접 설정
13.2	Pod 내부 권한	Pod 작성자	securityContext 직접 설정
13.3	Pod 생성 정책	클러스터 (PSA)	Namespace 라벨로 강제

그러나 Pod 하나만 단단해서는 부족합니다

아무리 Pod 자체를 잠궜도, Pod들이 서로 무제한으로 통신할 수 있다면 한 Pod가 뚫렸을 때 옆의 DB·API 서버까지 그대로 노출됩니다.

→ 13.4 NetworkPolicy로 "Pod 사이의 통로"를 제어

Section 13.4

Pod Network 격리 (NetworkPolicy)

Pod와 Pod 사이의 방화벽 — Ingress · Egress

13.1~13.3은 Pod 자체, 13.4는 Pod 사이

시선의 확장 — 점에서 선으로

13.1

Pod ↔ Node

↓ "근데 Node 접근을 막아도 컨테이너 안에서 root면 다 할 수 있잖아"

13.2

Pod 내부 권한

↓ "근데 작성자가 자물쇠 안 걸면 결국 다 무용지물이잖아"

13.3

Pod 생성 정책

↓ "근데 Pod 자체를 잠가도 Pod끼리 다 통신되면 옆 Pod로 옮겨가잖아"

13.4

Pod ↔ Pod 통신

출처: kubernetes.io/docs/concepts/services-networking/network-policies/

13.4의 출발점 — 기본 Network는 "벽이 없다"

기본값 — 모든 Pod는 서로 통신할 수 있다

Namespace가 달라도, 용도가 전혀 달라도 마찬가지.

■ 비유 — 칸막이 없는 사무실 (open floor)

누구나 누구의 자리에든 걸어가 말을 걸 수 있음.
편하지만, 악의적인 Pod 하나가 침투하면 옆자리·뒷자리 DB까지 마음껏 접근.

NetworkPolicy — 칸막이와 출입 규칙

라벨 셀렉터로 누가 누구에게 말 걸 수 있는지(인바운드),
어디로 나갈 수 있는지(아웃바운드)를 지정하는 방화벽 규칙.

13.4.1 default deny — Network 격리 켜기

podSelector로 "대상 Pod"를 고르는 순간 격리 시작

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

핵심 원리

- 빈 podSelector {} = Namespace의 모든 Pod
- policyTypes에 Ingress만 → 허용 규칙 없음 = 모두 차단
- "명시적으로 허용된 트래픽 외엔 모두 차단" (default deny)

⚠ 예외 — Node에서 오는 트래픽은 항상 허용

kubelet이 보내는 헬스 체크(liveness-readiness probe)가 막히면 Pod가 곧바로 재시작됨. NetworkPolicy의 의도된 동작.

13.4.2 특정 Pod만 접속 허용

라벨 셀렉터 기반 화이트리스트

```
kind: NetworkPolicy
metadata:
  name: postgres-netpolicy
spec:
  podSelector:
    matchLabels:
      app: database      # 보호 대상
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: webserver  # 이 라벨만 허용
      ports:
        - port: 5432
```

효과

DB 금고실에는 webserver
사원증을 가진 사람만,
5432번 문으로만 들어올 수 있다.

프론트엔드 Pod가 침해당해도
DB에 직접 붙을 수 있는 건
정해진 Pod뿐 →
피해 확산 차단

13.4.3 Namespace 간 격리

namespaceSelector로 다른 Namespace 통제

```
ingress:
- from:
  - namespaceSelector:
      matchLabels:
        tenant: manning
```

podSelector vs namespaceSelector

podSelector - 같은 ns에서 누구나
namespaceSelector - 어느 ns에서 왔냐

⚠ AND vs OR 함정 — 가장 많이 틀리는 부분

AND (같은 - 항목)

```
- namespaceSelector:
  matchLabels:
    name: monitoring
podSelector:
  matchLabels:
    app: prometheus
```

OR (다른 - 항목)

```
- namespaceSelector:
  matchLabels:
    name: monitoring
- podSelector:
  matchLabels:
    app: prometheus
```

13.4.4 ipBlock — CIDR 표기법으로 격리

Pod 라벨이 아닌 IP 대역으로 허용·차단

```
ingress:
- from:
  - ipBlock:
      cidr: 172.17.0.0/16
    except:
      - 172.17.1.0/24
```

except 필드

허용 CIDR 범위에서
특정 서브넷만 골라 차단.
"이 동네는 OK, 이 골목길만 출입 금지"

⚠ Node IP는 ipBlock과 무관하게 항상 허용

13.4.1의 default deny 예외와 같은 이유 — 헬스 체크 때문.

ipBlock: 192.168.1.0/24 처럼 Node IP가 포함되지 않는 범위를 지정해도,
Pod가 떠 있는 Node의 IP에서 오는 트래픽은 차단되지 않습니다.

→ 사용처: 클러스터 밖 레거시 서버나 특정 게이트웨이의 고정 IP 허용

13.4.5 Egress — 아웃바운드 제한

Pod가 어디로 나갈 수 있는지 제어

```
kind: NetworkPolicy
metadata:
  name: egress-netpolicy
spec:
  podSelector:
    matchLabels:
      app: webserver
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - port: 5432
```

▣ 비유

인바운드 — "누가 우리 집에 들어올 수 있나"

이그레스 — "우리 집 사람은 어디로 외출할 수 있나"

왜 중요한가

Pod 탈취 후 외부 공격 서버로
데이터 유출(exfiltration) 방지

13장 전체 정리 — 두 겹의 방어선이 완성

안(Pod 자체)과 사이(Pod 간 통신) 두 방향

절	통제 대상	한 줄 요약
13.1	호스트 Namespace	Pod가 Node 자원에 얼마나 손대는가
13.2	보안 컨텍스트	컨테이너가 무슨 권한으로 실행되는가
13.3	PSA	위험한 설정을 클러스터가 강제로 통제
13.4	NetworkPolicy	Pod 사이의 통신을 누가 허용받는가

💡 실무 권장 — 제로 트러스트 접근

운영 클러스터에서는 먼저 default-deny(모두 차단) 정책을 깔고,
꼭 필요한 통신만 하나씩 화이트리스트로 여는 방식이 가장 안전합니다.

Thank You

Q & A

Chapter 13 · 클러스터 Node와 Network 보안